

EXERCISE I

With reference to the back-propagation algorithm

I.a) Explain the concept of learning rate, provide a formula where it is used for updating the weights of the neural network

Learning rate (λ) is the hyper-parameter that defines the adjustment in the weights of our network with respect to the loss gradient descent. It determines how fast or slow we will move towards the optimal weights

The Gradient Descent Algorithm estimates the weights of the model in many iterations by minimizing a cost function at every step.

At every step, the network's weights gets updated according to this formula

$$W = W - \lambda * \partial F(W) / \partial W$$

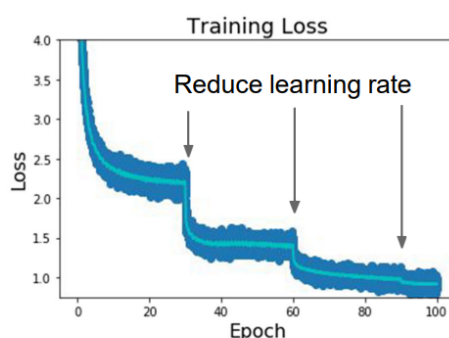
Where:

- W is the weight
- λ is the learning rate
- $F(W)$ is the cost function.

I.b) Explain the concept of learning rate scheduling in the context of SGD.

The Learning rate scheduler is in charge of adjusting the learning rate between epochs or iterations as the training progresses.

Optional: three examples of learning rate scheduling are: the cosine, linear, and inverse square root. Formulas below :



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2} \alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0 (1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0 / \sqrt{t}$

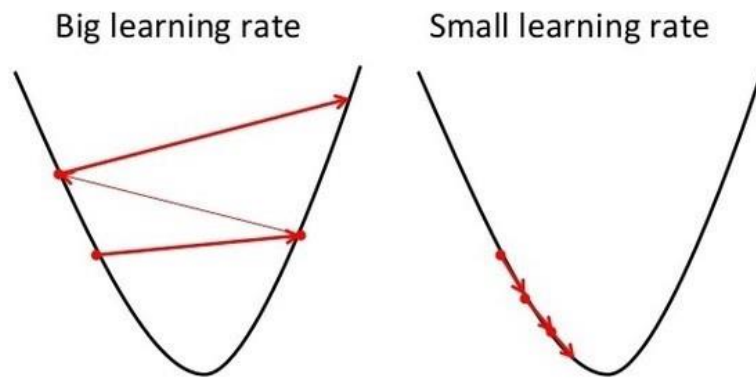
α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

I.c) Why for most schedulers the learning rate is a monotonically decreasing function? Give an intuitive explanation

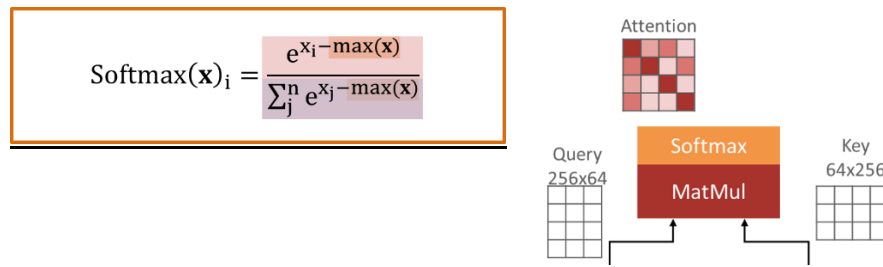
A small learning rate may result in a long training process that could get stuck in local minima, whereas a large learning rate may result in an unstable training process, never reaching a local or global minima.



Therefore the learning rate is set to be large at the beginning of the training process, so to have an initial fast convergency, and then it is gradually decreased with the training epochs ensuring to reach the global minima of the loss function.

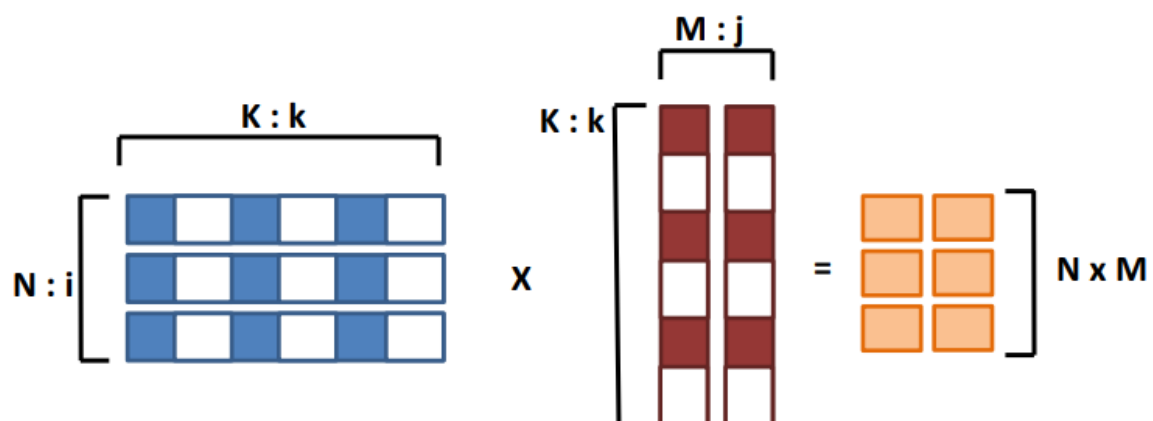
EXERCISE II

Consider the attention layer $\text{Softmax}(QK)$ as depicted above with the sizes of the Q and K matrices shown in the picture. The softmax is computed over each row of the QK matrix. For the QK product calculation assume we use the straightforward algorithm based on row by column dot products.



Where $\mathbf{x}$ is the row vector, i and j are iterators over the row elements

II.a) Compute the total number of **MAC** operations needed to compute QK



The number of MACs for a MatMul are:

$$\text{MACs} = N \times K \times M$$

In our case:

- $N = 256$
- $K = 64$
- $M = 256$

$$\text{MACs} = 256 \times 64 \times 256 = 4\,194\,304$$

II.b) Compute the total number of operations required to compute the Softmax, consider exponentiation and square root as elementary operations.

After the matmul, the output matrix has a size of AxB with A=256 and B=256

$$\text{Softmax}(x)_i = e^{x_i - \max(x)} / \sum_j (e^{x_j - \max(x)})$$

Where x is the QK matrix

Lets calculate the total number of operations needed :

1. Find max of each row of the matrix QK. The matrix has 256x256= 65536 elements

Compare_ops_per_row= 256-1 = 255COMP

TOT_compares= 255 COMP *256 = 65280 COMP

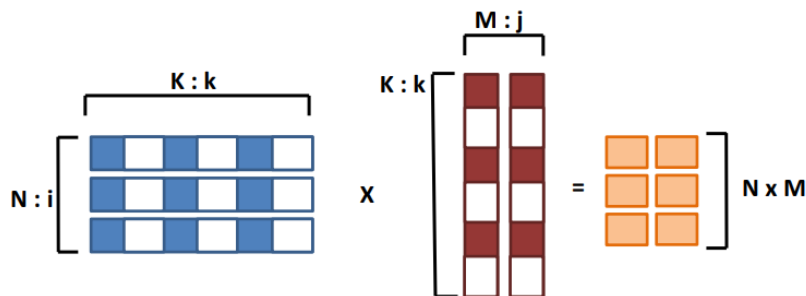
2. Normalization: we calculate x-max(x) for each element of the x matrix

This operation bounds the exponent of exponentials to avoid numerical problems: the exponent is always negative, while the exponential is always <1. As it is a multiplier of both dividend and divisor, it does not change the result.

This normalization requires:

MAC_norm = 256*256 ADDs = 65536 ADDs

3. number of operations of the softmax



Pseudo-code:

```
for(i=0; i< 256; i++){ //iterates over the rows
    for(j=0; j< 256; j++){ //iterates over the elements of the row i
        Denominator = sum(Exp(x_j));
    }

    for(j=0; j< 256; j++){ //iterates over the elements of the row I
        Numerator = Exp(x_j)
        Softmax(x) = Divide(Numerator, Denominator)
    }
}
```

- 256*256 sums (denominator)

- 256 * 256 exp (denominator)
- 256 exp (numerator)
- 256 divides (Softmax(x_i))

The total number of operations is then:

Ops = max(QK) + normalization + softmax =

(65280 COMP) + (65536 ADDs) + (256*256 SUM + 256*256 EXP + 256 EXP + 256 DIV) =

65280 + 65536 + 256*256 + 256*256 + 256 + 256 = **262400 Ops**

II.c) Compute the number of memory accesses (reads, writes) needed to compute Softmax(QK). Justify your answer

We provide here a slightly different solution, where we calculate:

1. The max of the row
2. The denominator of the softmax
3. The softmax

```
for row in range(n_rows):
    // find max
    for i in range(len(row)):
        read(xi)
        max := comp(xi, max)
write(max) //hypothesis: kept in memory

    // calculate denominator
max = read(max) //hypothesis: kept in memory
    denominator = 0
    for i in range(len(row)):
        xi = read(xi)
        denominator = add(denominator, exp(xi-max))
write(denominator) //hypothesis: kept in memory

    // calculate Softmax
denominator = read(denominator) //hypothesis: kept in memory
max = read(max) //hypothesis: kept in memory
    for i in range(len(row)):
        xi = read(xi)
        softmaxi = exp(xi-max)/denominator
        write(softmaxi)
}
```

1. Max computation:

Reads = $256 * 256$

~~Writes = 256 (max) // We assume that we don't keep the max(x) in processor's register.~~

2. Denominator:

~~Reads = 256 (max) // We assume that we don't keep the max(x) in processor's register.~~

Reads = $256 * 256 (x_i)$

~~Writes = $256 * 256$ (denominator) // We assume that we don't keep the max(x) in processor's register.~~

3. Softmax:

~~Reads = 256 (denominator)~~

~~Reads = 256 (max)~~

Reads = $256 * 256 (x_i)$

Writes = $256 * 256$ (softmax)

Total reads = $256 * 256 + 256 * 256 + 256 * 256 = 196608$

Total writes = $256 * 256 = 65536$

EXERCISE III

A parallel cluster of RISC-V processors is computing a sequence of two 5x5 convolutional layers, each with RELU and max-pooling (3x3 window). The input spatial size is 32x128. The first layer takes as an input 32 channels and outputs 60 channels. The second layer produces 30 output channels. Inputs and outputs are quantized to 8 bits and weights are quantized to 4 bits. Padding with zeros is used on the convolutions. Assume that on average a RISC-V processor can perform a 8-bit, 4-bit MAC every cycle, a ReLU every 3 cycles and a compare every 2 cycles. Assume that the computation is parallelized assigning output channels to be computed to processors, trying to maximally balance the workload. After each sub-layer (CONV, RELU, MAXPOOL, CONV, RELU, MAXPOOL), the processors synchronize (join) with a barrier

III.a) Compute the minimal memory needed to store the entire double-layer in L1. You can assume that the RELUs are computed in place.

Sequence: Input – Conv1 – Relu1 – maxpool1 – conv2 -relu2 - maxpool 2

Parameters=

- $\text{in_conv1} = w * h * ch = 32 * 128 * 32 = 131072$
- $\text{weights_conv1} = ch_in * ch_out * K * K = 32 * 60 * 5 * 5 = 48000$
- $\text{out_conv1} = w * h * ch = 32 * 128 * 60 = 245760$
- $\text{out_pool1} = w * h * ch = 30 * 126 * 60 = 226800$
- $\text{weights_conv2} = ch_in * ch_out * K * K = 60 * 30 * 5 * 5 = 45000$
- $\text{out_conv2} = w * h * ch = 30 * 126 * 30 = 113400$
- $\text{out_pool2} = w * h * ch = 28 * 124 * 30 = 104160$

$$\begin{aligned} \text{Total_L1} &= (\text{weights_conv1} + \text{weights_conv2}) * 0.5B + \\ &\quad + (\text{in_conv1} + \text{out_conv1} + \text{out_pool1} + \text{out_conv2} + \text{out_pool2}) * 1B = \\ &= (48000 + 45000) * 0.5B + (131072 + 245760 + 226800 + 113400 + 104160) * 1B = \mathbf{867692\ B} \end{aligned}$$

III.b) Compute the number of cycles to process the two layers on a single processor.

A maxpool needs

- $\text{Compare_per_pooling_window} = (3 * 3 - 1) = 8$
- $\text{Compare_per_1_output_channel} = 8 * W_{out} * H_{out}$
- $\text{Compare_for_all_output_channel} = 8 * W_{out} * H_{out} * C_{out}$

Ops(1core)=

- $\text{conv1} = K * K * C_{in} * C_{out} * W_{out} * H_{out} \text{ MAC} = 5 * 5 * 32 * 60 * 32 * 128 \text{ MAC} = 196608000 \text{ MAC}$
- $\text{Relu1} = C_{out} * W_{out} * H_{out} \text{ MAC} = 60 * 32 * 128 \text{ RELU} = 245760 \text{ ReLU}$
- $\text{pool1} = 8 * W_{out} * H_{out} * C_{out} \text{ CMP} = 8 * 60 * 30 * 126 \text{ CMP} = 1814400 \text{ COMP}$

- $\text{conv2} = K * K * \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout}$ MAC = $5 * 5 * 60 * 30 * 30 * 126$ MAC = 170100000 MAC
- $\text{Relu2} = \text{Cout} * \text{Wout} * \text{Hout}$ MAC = $30 * 30 * 126$ RELU = 113400 ReLU
- $\text{Pool2} = 8 * \text{Wout} * \text{Hout} * \text{Cout}$ CMP = $8 * 30 * 28 * 124$ CMP = 833280 COMP

$$\text{Cycles} = (\text{conv1} + \text{conv2}) * 1 + (\text{Relu1} + \text{Relu2}) * 3 + (\text{pool1} + \text{pool2}) * 2 =$$

$$(196608000 + 170100000) * 1 + (245760 + 113400) * 3 + (1814400 + 833280) * 2 = 373080840$$

III.c) Compute the speedup compared to sequential when the cluster has 8 cores.

Ops(1core)=

- $\text{conv1} = K * K * \text{Cin} * \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout}$ MAC = $5 * 5 * 32 * (8) * 32 * 128$
MAC = 26214400 MAC
- $\text{Relu1} = \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout}$ MAC = $(8) * 32 * 128$ RELU = 32768 ReLU
- $\text{pool1} = 8 * \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout}$ CMP = $8 * (8) * 30 * 126$ CMP = 241920 COMP
- $\text{conv2} = K * K * \text{Cin} * \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout}$ MAC = $5 * 5 * 60 * (4) * 30 * 126$
MAC = 22680000 MAC
- $\text{Relu2} = \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout}$ MAC = $(4) * 30 * 126$ RELU = 15120 ReLU
- $\text{Pool2} = 8 * \text{Wout} * \text{Hout} * \text{round_ceil}(\text{Cout}/8)$ CMP = $8 * (4) * 28 * 124$ CMP = 111104 COMP

$$\text{Cycles} = (\text{conv1} + \text{conv2}) * 1 + (\text{Relu1} + \text{Relu2}) * 3 + (\text{pool1} + \text{pool2}) * 2 =$$

$$(26214400 + 22680000) * 1 + (32768 + 15120) * 3 + (241920 + 111104) * 2 = 49744112$$

$$\text{Speedup} = 373080840 / 49744112 = 7.5$$